

# Relatório: Modificações no código “Gerador de Relatórios”

## Exercício-Programa 2

ACH2003 - Computação Orientada a Objetos | Profº Dr. Daniel Cordeiro

---

Luiza de Jesus Lopes - N°USP: 12822076

### Introdução

O presente relatório tem como objetivo relatar sucintamente as modificações realizadas no sistema “Gerador de Relatórios” programado em Java. O objetivo geral é refatorar o código com o emprego dos padrões de projeto Strategy e Decorator e modernizar o código para que ele use o arcabouço de coleções de Java. Como resultado final, é esperado que o sistema continue funcionando normalmente, mas com as devidas mudanças implementadas da maneira mais eficiente possível, conforme instruções.

### Desenvolvimento

Algo importante na refatoração de qualquer projeto é inicialmente entender bem sobre o problema e organizar a execução que se pretende executar. Para este projeto, primeiro li com cuidado e atenção sobre o que eu deveria fazer e dividi o problema inteiro em 4 partes: “Aplicando o padrão Strategy para resolver o problema de expansão”, “Transformando a lógica de vetores para listagem dos produtos para coleções de elementos”, “Definindo cada instância de produto com suas opções de formatação especificada individualmente utilizando o padrão Decorator” e “Modificando o código para carregar produtos de um arquivo .csv e salvar na coleção de produtos”. Após isso, procurei analisar o código e compreender como a dinâmica toda estava funcionando para então começar.

#### 1. Aplicando o padrão Strategy para resolver o problema de expansão

O código da classe *GeradorDeRelatorios* em sua forma inicial não permite que haja uma expansão de funcionalidades inteligente e flexível, como o próprio enunciado do EP diz, “eventuais extensões de funcionalidade demandam modificações em partes prontas do sistema e em diversos pontos do código”, como exemplo temos as classes *particiona* e *ordena* que possuem diversos *if-else-if*. Portanto, a primeira tarefa é eliminar essas condições ineficientes e aplicar o padrão Strategy para tornar as ações de ordenação e filtragem mais flexíveis e fáceis de entender.

Assim, para resolver o problema atrelado à ordenação, foram criadas as interfaces *CriterioOrdenacao* e *AlgoritmoOrdenacao* para serem, respectivamente, um “contrato” para novas classes que implementarem o método *compara* e para novas classes que implementarem o método *ordena*. Tais classes são:

- *CriterioDescricaoCrescente*: Implementa a interface *CriterioOrdenacao* e usa o método *compara*, recebendo 2 produtos e comparando a descrição deles;
- *CriterioPrecoCrescente*: Implementa a interface *CriterioOrdenacao* e usa o método *compara*, recebendo 2 produtos e comparando o preço deles;
- *CriterioEstoqueCrescente*: Implementa a interface *CriterioOrdenacao* e usa o método *compara*, recebendo 2 produtos e comparando a quantidade de estoque deles;

- QuickSortStrategy: Implementa a interface *AlgoritmoOrdenacao* e usa o método *ordena*, recebendo uma coleção de produtos e o critério de ordenação (QuickSort). A grande utilidade desta classe é que ela utiliza a mesma lógica de ordenação do código antigo, mas agora sem os vários *if-else-ifs*, que foram substituídos por *while()*, e sem se preocupar com qual critério estão usando porque chamam o método *critério.compara()*;
- InsertionSortStrategy: Implementa a interface *AlgoritmoOrdenacao* e usa o método *ordena*, recebendo uma coleção de produtos e o critério de ordenação (InsertionSort). Esta classe também possui a mesma lógica do código antigo, a mudança está na utilização do *critério.compara()* e, claro, na não utilização dos vários *if-else-ifs*.

Após a implementação das classes, foram removidas as constantes de algoritmo e critério da classe principal (*GeradorDeRelatorios*), assim como as variáveis *String algoritmo* e *String criterio*, e adicionadas as variáveis de instância *AlgoritmoOrdenacao algoritmoOrdenacao* e *CriterioOrdenacao criterioOrdenacao*. Além disso, o construtor também foi alterado para o novo contexto e os métodos *particiona* e *ordena* foram retirados do código. Foi criado apenas um novo método *ordena* mais simples do que o outro, que delega a tarefa para o algoritmo de ordenação que foi escolhido.

Ademais, foi adicionado a chamada *ordena()* dentro do método *geraRelatorio*.

Agora, para resolver o problema de filtragem, foi adotada a mesma estratégia, mas agora com apenas a interface *CriterioFiltragem* cujas novas classes que a assinarem terão de usar o método *ehSelecioneado* que recebe um produto *p* a ser verificado e retorna *true* se o produto deve ser incluído no relatório e *false* caso contrário. Tais classes são:

- FiltroTodos: Implementa a interface *CriterioFiltragem* e retorna todos os produtos selecionados;
- FiltroEstoqueMenorIgual: Implementa a interface *CriterioFiltragem* que recebe um valor limite através do método *FiltroEstoqueMenorIgual* e retorna a quantidade no estoque;
- FiltroCategorialigual: Implementa a interface *CriterioFiltragem* que recebe um valor limite através do método *FiltroCategorialigual* e retorna a quantidade nesta categoria específica.

Após a implementação da interface e novas classes, as devidas alterações na classe principal foram implementadas.

## 2. Transformando a lógica de vetores para listagem dos produtos para coleções de elementos

É explicitado no enunciado do EP e nítido no código também que eram utilizados vetores para armazenar a listagem dos produtos, isso claramente é uma maneira pouco flexível de se usar, portanto, foram trocados os arrays para o uso de coleções substituindo *Produto[]* por *List<Produto>* e trocando os métodos necessários de *List* para cada situação, isso dentro da classe principal no construtor e nos métodos *debug()*, *geraRelatorio()* e *carregaProdutos()*, assim como nas classes e interfaces que utilizavam o vetor de produtos.

Um ponto talvez importante de dizer é que, ao implementar esta mudança, me esqueci que teria que implementar uma forma do código ler um arquivo.csv para listar os produtos, então apliquei as mudanças sem considerar isto e tive que desconsiderar algumas alterações posteriormente.

### **3. Definindo cada instância de produto com suas opções de formatação especificada individualmente utilizando o padrão Decorator**

Para esta etapa, o objetivo era retirar esta funcionalidade da classe principal e definir a formatação dos produtos individualmente utilizando o padrão Decorator. Para isso, foi criada a classe abstrata *ProdutoDecorator* que implementa a interface *Produto* mas, por padrão, apenas repassa todas as chamadas de método para *Produto* que está formatando. Também foram criadas as classes *NegritoDecorator* e *ItalicoDecorator* que estendem *ProdutoDecorator* e que, respectivamente, formata para negrito e itálico utilizando as tags `<strong>` e `<em>` ao redor do resultado.

As devidas mudanças necessárias foram feitas na classe principal e em suas intra-classes *geraRelatorio* e *main*, assim, a classe principal agora só sabe que o produto tem um método *formataParaImpressao()* e que vai imprimir o que esse método retornar, já no *main* as mudanças estão relacionadas na responsabilidade de montar a cadeia de decoradores pegando um produto e, se a opção for negrito, embrulha com *NegritoDecorator*, e se a opção for itálico, então o resultado vem com *ItalicoDecorator*. Assim, o código se torna ainda mais flexível e exerce a responsabilidade única.

### **4. Modificando o código para carregar produtos de um arquivo .csv e salvar na coleção de produtos**

O objetivo era modificar o método *carregaProdutos()* que está dentro da classe principal para ler os dados de um arquivo .csv ao invés de criá-los manualmente como estava sendo. Para isso, foram feitas as novas importações, respectivamente, para ler arquivo linha a linha, ler dados de um arquivo e para lidar com potenciais erros: *java.io.BufferedReader*, *java.io.FileReader* e *java.io.IOException*. Posteriormente, foi criada a classe *CorDecorator* que estende *ProdutoDecorator* e formata a impressão para ser em itálico ou negrito, a diferença deles para as outras classes de negrito e itálico é que ele recebe um parâmetro extra no construtor que é a cor para gerar a tag HTML `<span style="color: ...">`. Então, dentro do método *geraRelatorio* na classe principal, foram deletadas todas as linhas que criavam os produtos manualmente, então, dentro de um bloco *try-catch* foi criado um loop que lê o arquivo linha a linha tendo a vírgula como separador, se os dados possuíam até 8 partes (id, descrição, categoria, qtdEstoque, preço, se é negrito, se é itálico e se há cor). Assim, chama-se os métodos necessários para cada tipo de formatação e lança exceções caso haja erros na leitura do arquivo.

### **Considerações finais**

Ao final das implementações das mudanças, percebe-se que o código continua funcionando normalmente, portanto, as novas mudanças foram implementadas com sucesso utilizando coleções, os padrões Strategy e Decorator e na importação de arquivos .csv. Além disso, o código também foi melhor comentado para fins de documentação.